Лекция 2. Классы и объекты. Статические данные. Конструктор, деструктор. Операции new и delete

Классы и объекты

Объединение данных и функций является центральной идеей объектно-ориентированного программирования. Как и структура, класс является инструментом создания нового типа данных, при этом, в отличие от структуры, класс включает в себя не только элементы-данные, но и элементы-функции.

Пример определения класса:

```
class SmallClass
{
private:
   int someData;            // поле данных класса
public:
   void setSomeData(int d)   // метод класса, изменяющий значение данных
   {someData = d;}
   int getSomeData()   // метод класса, возвращающий значение данных
   {return someData;}
   void showData()   // метод класса, отображающий значение данных на консоли
   {std::cout << “Data value is “ << someData << “\n”;}
}
```

Тело класса в примере содержит поле данных *someData* и три функции *setSomeData(), getSomeData()* и *showData()*. В классах всегда указываются спецификаторы доступа *private* и *public*, реализующие концепцию инкапсуляции. Если необходимо защитить какие-либо данные и функции, их помещают внутрь класса с ключевым словом *private.* Такие данные доступны только из функций-членов класса. Данные и функции, описанные с ключевым словом *public* доступны из любых функций программы. Сокрытие данных (инкапсуляция) применяется для того, чтобы оградить данные от использования в тех частях программы, в

которых нет необходимости использовать эти данные. Сокрытие данных позволяет уберечь программистов от случайных ошибок.

В приведенном примере функции определены непосредственно внутри класса. Такие методы класса называются встраиваемыми. Можно определить функции и вне тела класса:

```
class SmallClass
{
private:
   int someData;
public:
   void setSomeData(int d);
   int getSomeData();
   void showData();
}

void SmallClass::setSomeData(int d)
{someData = d;}

int SmallClass::getSomeData()
{return someData;}

void SmallClass::showData()
{std::cout << “Data value is “ << someData << “\n”;}
```

В приведённом примере перед именем функциями стоит имя класса *SmallClass* и символ :: . Символ двойного двоеточия называется операцией разрешения области действия. Эта операция устанавливает связь функции и класса, к которому относится функция. В данном случае запись *SmallClass::setSomeData(int d)* означает, что функция *setSomeData(int d)* является методом класса *SmallClass*.

Создание переменных, то есть объектов класса, производится также, как и создание структурных переменных:

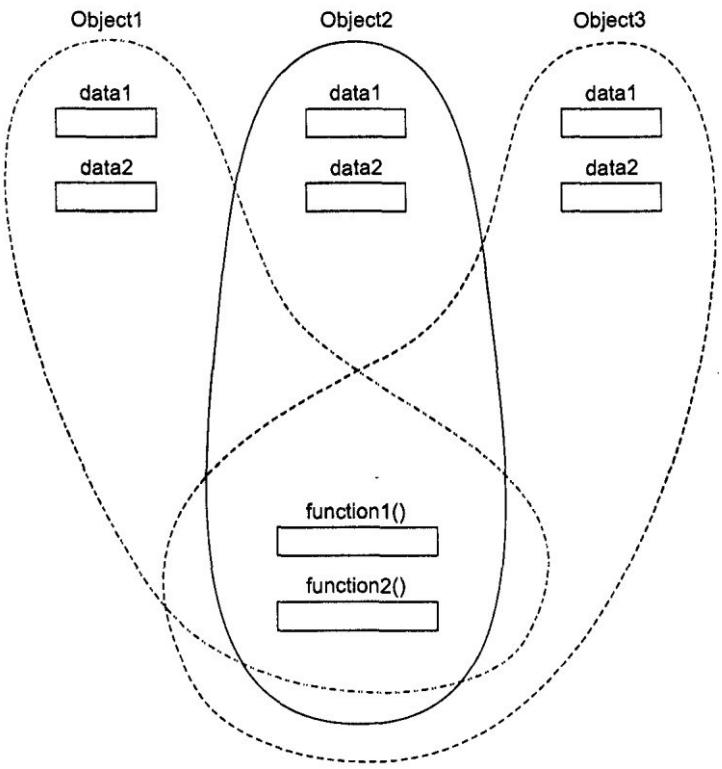
© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

SmallClass s1, s2;

Обращение к элементам-данным и элементам функциям объекта производится с помощью операций «точка» и «стрелка».

s1.setSomeData(36);

Каждый объект имеет собственные независимые поля данных, но все объекты одного класса используют одни и те же методы. Методы класса помещаются в память компьютера всего один раз – при создании класса. Этот принцип проиллюстрирован на схеме.



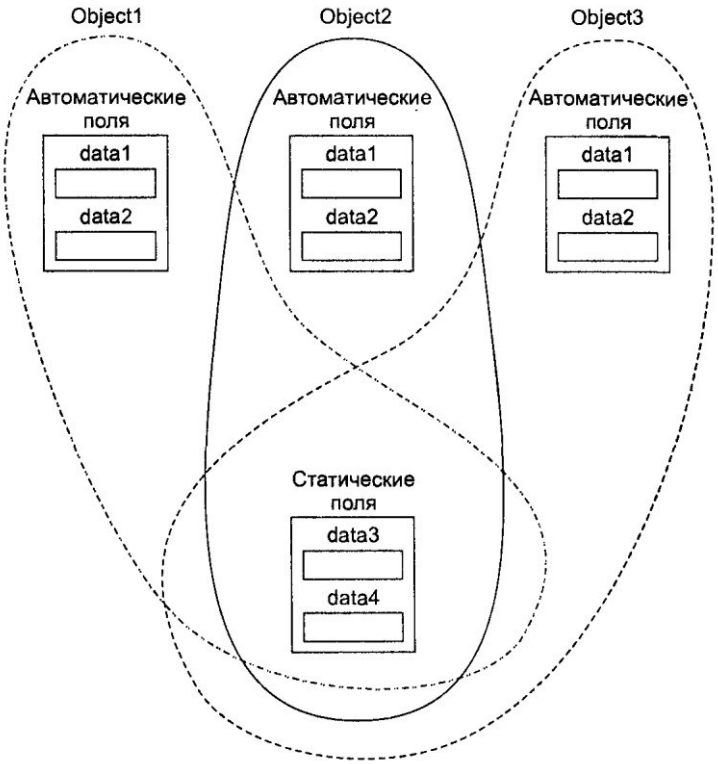
Любая функция-элемент класса имеет доступ к объекту, для которого она вызвана, через ключевое слово *this*. *this* – это указатель на переменную, которая вызвала функцию.

Статические функции и данные классов

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

Если поле данных класса описано с ключевым словом *static*, то значение этого поля будет одинаковым для всех объектов данного класса. Статические данные класса полезны в тех случаях, когда необходимо, чтобы все объекты включали в себя какое-либо одинаковое значение. Статическое поле по своим характеристикам схоже со статической переменной, оно видимо только внутри класса, но время его жизни совпадает со временем жизни программы. Таким образом, статическое поле существует даже в том случае, когда не существует ни одного объекта класса. Тем не менее, в отличие от статической переменной функции, предназначенной для сохранения значения между вызовами, статическая переменная класса используется для хранения данных, совместно используемых объектами класса.

Свойства статических полей класса обусловлены особым способом их хранения в памяти. Если обычные поля являются независимыми у различных объектов (переменных), то статические поля расположены у различных объектов класса в одной ячейке памяти (см. рисунок).



© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

В этом отношении можно увидеть аналогию статического поля класса с глобальной переменной. Помимо объявления в определении класса, статический элемент данных должен еще и *определяться:*

```
class SomeClass
{
   static int iCount;   // Объявление статического элемента
};

int SomeClass::iCount = 0;   // Определение статического элемента
```

Обращаться к открытым статическим элементам класса можно либо через любой его представитель операциями точка и стрелка, либо с помощью операции разрешения области действия *SomeClass::iCoun*t. Последний способ предпочтительнее, так как ясно показывает, что элемент не связан с конкретным объектом.

Если со спецификатором *static* объявляется элемент-функция, то она не связывается ни с какими представителями класса, то есть в ней не существует указатель *this*. Статическая функция-элемент класса может вызываться, даже если никаких представителей класса не существует и может обращаться только к статическим элементам-данным класса и вызывать только другие статические элементы-функции класса.

Константные функции классов

Константные методы отличаются тем, что не изменяют значений полей своего класса. Для того, чтобы сделать функцию константной, необходимо указать ключевое слово *const* после прототипа функции, но до начала тела функции (как в примере ниже) – как при объявлении функции, так и при определении, если они разделены. Те методы, которые лишь считывают данные из поля класса, имеет смысл делать константными, поскольку у них нет необходимости изменять значения полей объектов класса. Использование константных

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

функций помогает компилятору обнаруживать ошибки и указывает, что функция не изменяет значений полей объектов. Пример объявления константного метода приведен ниже.

```
class AClass
{
private:
   int alpha;
public:
   void baseFunc()   // не константный метод
   {alpha = 99;}     // корректно
   void conFunc() const   // константный метод
   {alpha = 99;}     // ошибка, этот метод не может менять значения полей
}
```

Для чего нужны константные методы? Если объект создается как типизированная константа (то есть объявляется с ключевым словом *const*), то он становится недоступным для изменения и для такого объекта можно вызвать только константные методы, поскольку только они гарантируют, что объект не будет изменён. Если же необходимо разрешить константной функции изменять некоторые элементы-данные у константных объектов, то их необходимо объявить с ключевым словом *mutable*.

```
class AnyClass
{
private:
   int value;
   mutable char msg;
public:
   jnt GetValue( ) const;
};

jnt AnyClass::GetValue( ) const
{
```

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

```
    msg =  10; // Допускается, поскольку msg - mutable.
   // value изменять нельзя:
   // value = 111;
   //
   return value;
}
```

Конструкторы и деструкторы

Конструктор – это метод класса, имя которого совпадает с именем класса и который выполняется каждый раз при создании нового объекта. Конструктор не возвращает значения, однако может принимать аргументы. Часто конструкторы применяются для инициализации создаваемых объектов класса. Конструкторы допускают перегрузку, поэтому возможна инициализация объекта несколькими способами. Имя конструктора в точности совпадает с именем класса, таким образом компилятор отличает конструктор от других методов. Конструктор *автоматически* вызывается всякий раз при объявлении переменной-объекта.

Одной из наиболее часто возлагаемых на конструктор задач является инициализация полей класса при объявлении переменной. Элементы данных класса часто инициализируют в теле конструктора, присваивая им соответствующие значения. Однако существует альтернативный механизм инициализации. Он использует *список инициализации элементов.*

Список инициализации следует за заголовком определения конструктора после двоеточия и состоит из *инициализаторов* элементов данных и базовых классов, разделенных запятыми. Каждому элементу списка передается один или несколько параметров, требуемых для инициализации.

Вот простейший пример класса с двумя перегруженными конструкторами, в одном из которых применяется обычный способ инициализации в теле функции, а во втором - список инициализации элементов:

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

```
class Time
{
private:
   int hr;
   int min;
public:
   Time(int h)
   { hr = h; min = 0;}
   Time(int h, int m): hr(h), min(m)
  { }
};
```

При объявлении в программе переменной как

Time h1(10);

будет вызван первый конструктор, и поля объекта *h1* получат значения 10 и 0 соответственно. При объявлении в программе переменной как

Time h2(10, 20);

будет вызван второй конструктор (хотя его тело и является пустым) и поля объекта *h2* получат значения 10 и 20 соответственно.

Список инициализации является единственным средством присвоения значений элементам данных класса, объявленных как *const*. Инициализация значений списком инициализации производится программой *до того*, как будет вызвана сама функция конструктор.

Существует также ещё одна специальная функция-элемент класса, вызываемая автоматически при создании переменной – конструктор копии. Он отличается от простого конструктора тем, что имеет один параметр – ссылку или константную ссылку на объект данного класса. Конструктор копии автоматически вызывается компилятором, когда вновь создаваемый объект инициализируется значением существующего объекта.

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

Пример:

```
class Time
{private:
   int hr, min;
public:
   Time(int h, int m): hr(h), min(m)
   { }                                  // Обычный конструктор
   Time(const Time &src) // Конструктор копии
   {
      hr = src.hr; min = src.min;
   }
};

int main()
(
   Time start (17,45); // Вызывается первый конструктор (обычный)
   Time current = start; // Вызывается конструктор копии
}
```

Если в классе конструктор копии не определен, компилятор генерирует конструктор копии по умолчанию, который производит простое копирование данных объекта в новую переменную. Если класс содержит какие-то указатели или ссылки, то такое копирование может быть бессмысленным или ошибочным. Поэтому иногда требуется запретить копирование объектов при создании. Если требуется запретить копирование объектов при их создании, то имеет смысл объявить конструктор копии (это может быть просто пустая функция) в закрытом разделе определения класса. Тогда другие функции не смогут создавать копии существующих объектов класса.

Деструктор является противоположностью конструктора. Эта функция вызывается автоматически при уничтожении объектов и по смыслу должна производить освобождение

занимаемой объектом памяти и разного рода ресурсов (например, закрывать используемые объектом файлы). Именем деструктора должно быть имя класса, которому предшествует символ ~. Деструктор не имеет параметров и не может возвращать значений.

Операции new и delete

Конструктор и деструктор служат для обеспечения создания и удаления переменных при их статическом размещении в памяти, то есть с помощью оператора объявления переменных. В том случае, если переменные размещаются в памяти динамически, роль конструктора и деструктора берут на себя операторы new и delete соответственно. По умолчанию они действуют как обычно – просто выделяют и освобождают память под объект. Переопределить для класса возможно 4 операции:

operator new( ) – вызывается при создании динамических объектов;

operator new[]( ) - вызывается при создании динамических массивов объектов;

operator delete( ) – вызывается при удалении динамических объектов;

operator delete[]( ) – вызывается при удалении массивов динамических объектов;

Пример переопределения операции new и delete:

```
class Hold
{
public:
   int *a; //указатель на массив
   void *operator new(size_t size); // Операция new класса.
   void operator delete(void *); // Операция delete класса.
};

void *Hold::operator new(size_t size)
{
   Hold *help;
   help = (Hold*)malloc(size); // выделяем память под объект
```

```
    help->a = (int*)malloc(10*sizeof(int)); // выделяем память под массив из 10 чисел
    return help;
}

void Hold::operator delete(void *p)
{
 free (((Hold*)p)->a); // удаляем память из-под массива
free (p); // Удаляем память из-под объекта
}
```

Вопросы к лекции 2

1. В определении класса члены класса с ключевым словом private доступны

   а) любой функции программы

   б) в случае, если известен пароль

   в) методам этого класса

   г) только открытым членам класса

2. Напишите определение класса studentgroup, включающего одно закрытое поле типа int с именем number и одним открытым методом с прототипом void add( );

3. Истинно ли следующее утверждение: поля класса должны быть закрытыми.

4. Для чего при работе с объектами применяется операция «точка»

5. Для чего при работе с объектами применяется операция «стрелка»

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

6. Что такое конструктор

7. Как задается имя конструктора?

8. Может ли класс иметь более одного конструктора?

9. Методу класса всегда доступны данные

   а) объекта, членом которого он является

   б) класса, членом которого он является

   в) любого объекта класса, членом которого он является

10. Что является единственным формальным различием между структурами и классами в C++?

11. Пусть определены три объекта класса. Сколько копий полей класса содержится в памяти? Сколько копий методов класса?

12. Константный метод, вызываемый для объекта класса

   а) может изменить как константные, так и неконстантные поля

   б) может изменить только неконстантные поля

   в) может изменять только константные поля

   г) не может изменять никакие поля

13. Для чего нужно объявление поля класса со словом mutable

14. Что такое деструктор

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

15. Какие свойства приобретает поле данных класса, объявленное как static

16. В чем состоит преимущество определения конструктора со списком инициализации элементов?

17. Для чего необходимо переопределять операции new и delete.

18. Какие свойства приобретает элемент-функция класса, если она объявлена как статическая

19. Что такое указатель this?

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС